

Enhancing Answers with External Knowledge/Private Documents

Practice 3 Transcript | Oct 1, 2026 15:38

David Chen 00:00

Glad you're here. Our AI assistant answers basic questions well, but it struggles when users ask about our company policies or product details. How do you think we can fix this gap?

You 00:28

We will resolve this knowledge gap by implementing a Retrieval-Augmented Generation architecture using Spring AI's `VectorStore` to index our internal documents and configuring a `QuestionAnswerAdvisor` to dynamically retrieve and inject relevant company policies directly into the prompt.

David Chen 00:31

That's a solid approach! Using a Retrieval-Augmented Generation setup could really help bridge the gap. With the `VectorStore`, you can index internal documents for fast similarity searches. Now, when configuring the `QuestionAnswerAdvisor`, how do you plan to handle cases where the retrieval doesn't yield relevant results? Any fallback strategies in mind?

You 02:14

We will set a strict similarity threshold on the `VectorStore` `SearchRequest` and implement a custom Spring AI `Advisor` that intercepts zero-retrieval results to return a deterministic fallback response or route the query to a human agent rather than allowing the LLM to speculate.

David Chen 02:16

That's a really thoughtful fallback strategy! Setting a similarity threshold ensures only meaningful results come through, and intercepting zero-retrieval cases avoids guesswork—great for maintaining trust. How are you planning to balance performance with accuracy, especially for larger document sets in the `VectorStore`? Indexing can get tricky at scale.

You 04:59

We will balance performance and accuracy for large document sets by tuning chunk sizes via Spring AI's `TokenTextSplitter`, applying `SearchRequest` metadata filters to constrain the retrieval search space, and leveraging vector database HNSW indexing for low-latency similarity queries.

David Chen 05:02

Absolutely brilliant! Using `TokenTextSplitter` to fine-tune chunking is key for optimal token usage, and metadata filters can really narrow down the search effectively. HNSW indexing is a great choice for keeping those latency numbers low, even with larger datasets. Are you planning on implementing any monitoring to track how well these optimizations hold up under load?

You 07:20

We will monitor system behavior under load by configuring Spring Boot Actuator and Micrometer to expose native Spring AI observability metrics, such as `db.vector.client.operation` latency and `gen_ai.client.token.usage`, which can be exported to Prometheus and visualized in Grafana. Monitor Spring AI Apps Like a Pro using Micrometer and Grafana This practical guide demonstrates how to configure these observability tools to track token usage and identify performance bottlenecks in Spring AI applications.

David Chen 07:23

That's fantastic! Leveraging Spring Boot Actuator and Micrometer for observability is a pro move, and exporting metrics to Prometheus with Grafana visuals will give you clear insights. Tracking token usage and latency like that will make it easier to spot issues early. Are you also planning any alerting mechanisms for when performance thresholds are breached?

You 10:39

We will implement Prometheus Alertmanager to define threshold-based alerting rules on exported Spring AI metrics, such as `gen_ai.client.token.usage` limits or `db.vector.client.operation` latency, and configure Grafana to route those notifications directly to our internal incident response channels.

Enhancing Answers with External Knowledge/Private Documents

David Chen 10:44

Smart setup! Using Prometheus Alertmanager for threshold-based alerts and routing them through Grafana to your incident response channels is a great way to stay proactive. It sounds like you've got a robust plan in place. Any other areas you're considering optimizing, or does this wrap up your strategy?

You 11:14

We will finalize the architecture by deploying Spring AI Privacy Guardrails to automatically tokenize PII before it leaves the application boundary, and integrating Spring AI TypeSafe structured judgments to continuously evaluate response quality against our defined rubrics.

David Chen 11:17

That's next-level planning! Adding Privacy Guardrails for tokenizing PII is a really responsible move, and using TypeSafe structured judgments to evaluate response quality will ensure the assistant consistently meets your standards. This architecture sounds rock-solid. Great work pulling all these elements together!

End of session
